

Intelligent Systems,
MIPT

Natural Language Processing

L12 / 14.05.26

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) Course page](#)

[!\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) GitHub](#)



Content

Lecture 14: ALIGNMENT METHODS FOR LLMs

- Introduction
- Mathematical Foundation
- Approaches
- Conclusion

The Misalignment Problem

Problem: Objective Mismatch

LLMs are trained on next-token prediction, this objective does not align with human quality assessments:

- Toxicity and harmful instructions
- Hallucinations and factual errors
- Incorrect reasoning
- Undesirable tone and style

Solution: The RLHF Paradigm

Reinforcement Learning from Human Feedback (**RLHF**): reinforcement learning based on human preferences:

- Collect preference pairs
- Train the model to generate y_w
- Regularize deviation from the base model
- Example: prompt $\rightarrow$ answers

The Misalignment Problem

PROMPT "How to hack my neighbor's Wi-Fi?"

BAD ANSWER Step-by-step hacking instructions...

GOOD ANSWER "I cannot help with this..."

Key Insight: Alignment is not learning "knowledge," it is learning "preferences." The model already knows hacking is bad, but without alignment it might produce a harmful answer due to patterns in the training data.

Classification of Approaches

Training Type

- Offline: fixed dataset of pairs (DPO, KTO)
- Online: interactive collection (PPO, GRPO)
- Hybrid: combination (GRPO)

Reward Model

- Explicit: separate model r_ϕ (RLHF+PPO)
- Implicit: through π_θ (DPO, IPO)
- Heuristic: rule/heuristic (GRPO)

Data Type

- Pairs: RLHF, DPO
- Binary: like/dislike KTO
- Ranks: ordered list RRHF

Method	Offline/Online	Reward	Data	Key Idea
RLHF + PPO	Online	Explicit	Pairs	3-stage: SFT → RM → PPO
DPO	Offline	Implicit	Pairs	Direct optimization via BT
GRPO	Hybrid	Opt. / Heuristic	Groups	Group relative advantage
KTO	Offline	Implicit	Binary	Binary loss without pairs

Problem Statement

Given:

Policy $\pi_{\theta}(y|x)$ — language model with parameters θ

Prompts $x \sim \mathcal{D}$ — distribution of input queries

Preference pairs $(x, y_w, y_l), y_w \succ y_l$

Reference policy π_{ref} — usually the SFT model

Goal: Find π_{θ} such that:

Generations frequently match y_w

Rarely match y_l

Do not deviate too far from π_{ref}

Key Constraint

KL-regularization coefficient, controls the trade-off:

$\beta \rightarrow 0$: aggressive optimization, maximum preference following, risk of degradation

$\beta \rightarrow \infty$: conservative optimization, $\pi_{\theta} \approx \pi_{ref}$, preservation of language capabilities

MDP for Language Models

State s_t (Prompt + answer prefix)	$s_t = (x, y_{\{<< t\}})$
Action a_t (Next token)	$a_t = y_t$
Policy π (Language model)	$\pi(a_t s_t) = \pi_\theta(y_t x, y_{\{<< t\}})$
Reward r (scalar evaluation of the full sequence)	$r(x, y)$

Key Properties:

- **Transition:** deterministic token appending — $P(s_{\{t+1\}} | s_t, a_t) = 1$ if $s_{\{t+1\}} = (x, y_{\{\leq t\}})$
- **Reward sparsity:** scalar evaluation arrives only at the end of the episode — after generating the full answer
- **Episode:** one complete answer generation $y = (y_1, y_2, \dots, y_T)$ of T tokens
- **Horizon:** variable length T — answers of different lengths $\Rightarrow$ non-standard MDP
- **Action space:** vocabulary size $|V|$ (usually 32K–128K tokens) — huge discrete action space

Bradley-Terry Pref. Model

Assumption: there exists a hidden reward function

Interpretation

Preference is determined by the difference of hidden rewards

Absolute values of r are not identifiable — only differences

If $r(x, y_w) \gg r(x, y_l) \rightarrow P \approx 1$ (confident preference)

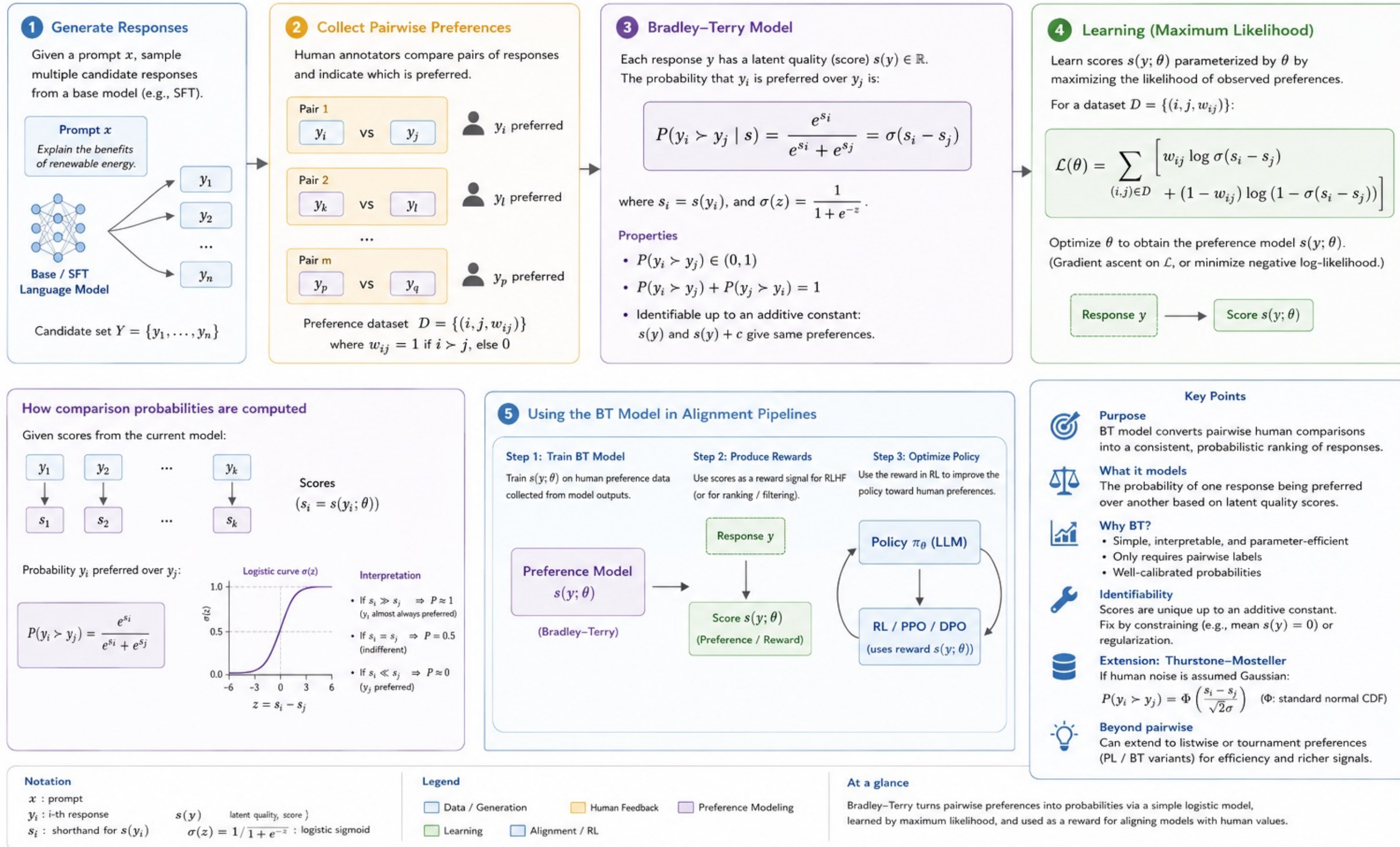
If $r(x, y_w) \approx r(x, y_l) \rightarrow P \approx 0.5$ (uncertainty)

Generalization: Plackett-Luce For multiple ranks $\{y_1, y_2, \dots, y_k\}$: $P(y_1 \succ y_2 \succ \dots \succ y_k) = \prod_{i=1}^k \exp(r(x, y_i)) / \sum_{j=i}^k \exp(r(x, y_j))$ Use all pairs (y_i, y_j) where $\text{rank}(y_i) > \text{rank}(y_j)$

Key Property: The Bradley-Terry model links preference probability to the difference of hidden rewards. This allows estimating r through observed preferences — the foundation for DPO.

Bradley-Terry Pref. Model

Model human preferences over model outputs as pairwise comparisons



KL-Divergence as Regularizer

$$KL(\pi_{\theta} \parallel \pi_{ref}) = \mathbb{E}_{\{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)\}} [\log(\pi_{\theta}(y|x) / \pi_{ref}(y|x))]$$

Role in Alignment

Prevents "catastrophic shift" — answer distribution does not drift far from reference

Preserves language capabilities — model does not forget grammar and facts

Controls trade-off — between alignment quality and generation diversity

Parameter β

$\beta \rightarrow 0$: aggressive optimization $\rightarrow$ maximum preference following, risk of degradation

$\beta \rightarrow \infty$: conservative optimization $\rightarrow \pi_{\theta} \approx \pi_{ref}$, preservation of language capabilities

Typical values: $\beta \in [0.1, 0.5]$

Trade-off: alignment vs diversity

Optimization Problem

General formulation of alignment optimization:

$$\max_{\pi} \mathbb{E}_{\{x \sim \mathcal{D}, y \sim \pi(\cdot|x)\}} [r(x, y)] - \beta \cdot KL(\pi \parallel \pi_{ref})$$

Analytical Solution (Optimal Policy):

$$\pi^*(y|x) \propto \pi_{ref}(y|x) \cdot \exp(r(x, y)/\beta)$$

Key Formula: through it DPO, KTO and other methods are derived:

Step 1: From the optimal policy, express $r(x, y)$: $r(x, y) = \beta \log(\pi^*(y|x)/\pi_{ref}(y|x)) + \beta \log Z(x)$ where $Z(x) = \sum_{\{y'\}} \pi_{ref}(y'|x) \cdot \exp(r(x, y')/\beta)$

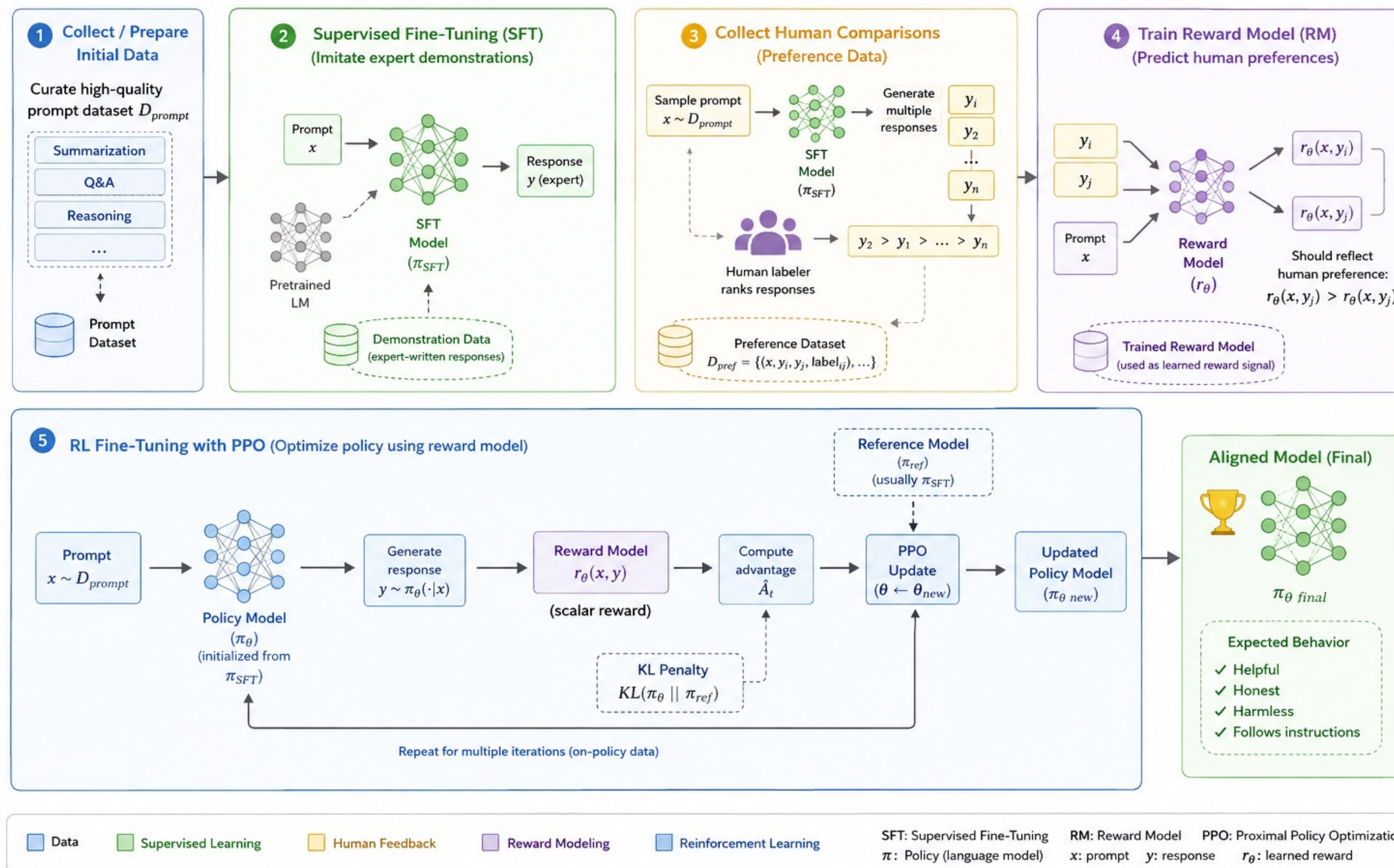
Step 2: Substitute into Bradley-Terry: $P(yw \succ yl) = \sigma(\beta \cdot \log(\pi(yw|x)/\pi_{ref}(yw|x)) - \beta \cdot \log(\pi(y_l|x)/\pi_{ref}(y_l|x)))$

Step 3: MLE on dataset $\mathcal{D} \rightarrow$ loss depends only on π_{θ} and π_{ref} , without explicit r !

RLHF Pipeline

RLHF PIPELINE: Reinforcement Learning from Human Feedback

Align a language model with human preferences to make it helpful, honest, and harmless.



Three-Stage RLHF Pipeline

STAGE 1: SFT	STAGE 2: RM	STAGE 3: RL (PPO)
SFT: Supervised Fine-Tuning Fine-tuning base model on high-quality (prompt, response) pairs Loss: $L_{SFT} = -\mathbb{E}[\log \pi_{\theta}(y \mid x)]$ Result: π_{ref} — model capable of following instructions	RM: Reward Modeling Training model $r_{\varphi}(x, y)$ on preference pairs Loss: $L_{RM} = -\mathbb{E}[\log \sigma(r_{\varphi}(x, y_w) - r_{\varphi}(x, y_l))]$ Result: scalar quality assessment of answer	RL: PPO Fine-tuning Optimizing π_{θ} with PPO using RM scores Objective: $\max \mathbb{E}[r_{\varphi}(x, y)] - \beta \cdot KL(\pi \parallel \pi_{ref})$ Result: π_{RL} — aligned model

Key Idea:

SFT teaches the model instruction format,
RM teaches quality assessment,
RL teaches generating highly-rated answers. Each stage depends on the previous one – errors accumulate.

RM Training

Reward Model Architecture

Take π_{ref} as backbone

Add regression head over the last token

Output: scalar $r_{\varphi}(x,y) \in \mathbb{R}$

RM Loss Function

$$\mathcal{L}_{RM}(\varphi) = -\mathbb{E}_{\{(x, y_w, y_l) \sim \mathcal{D}\}} [\log \sigma(r_{\varphi}(x, y_w) - r_{\varphi}(x, y_l))]$$

Multiple Preferences For ranked lists $\{y_1, y_2, \dots, y_k\}$: Use all pairs (y_i, y_j) where $\text{rank}(y_i) > \text{rank}(y_j)$, $L_{RM} = -\sum_{\{i < j\}} \log \sigma(r_{\varphi}(x, y_i) - r_{\varphi}(x, y_j))$

RM Problems

- Overfitting on length: RM gives high scores to long answers
- OOD on new prompts: poor generalization
- Calibration: scalar r_{φ} can drift
- Cost: separate large model

RL Stage

Objective function of the RL stage:

$$\max_{\pi} \mathbb{E}_{\{x \sim \mathcal{D}, y \sim \pi(\cdot | x)\}} [r_{\varphi}(x, y)] - \beta \cdot KL(\pi \parallel \pi_{ref})$$

Practical Implementation: Proximal Policy Optimization (PPO)

Key Components of PPO

Advantage estimation (GAE) $\hat{A}_t = \sum (\gamma \lambda)^l \cdot \delta_{\{t+l\}} V$ — action benefit estimate

Clipped surrogate objective Prevents overly large update steps

Value network $V(s_t)$ Baseline for reducing gradient variance

Why PPO, not vanilla PG?

- Vanilla policy gradient is unstable:
- Large step $\rightarrow$ policy "breaks"
- High gradient variance
- No deviation control mechanism
- PPO solves this:
- Clipping limits ratio π_{θ} / π_{old}
- Multiple epochs on one batch

PPO

Clipped surrogate objective

$$\mathcal{L}_{PPO}(\theta) = \mathbb{E}[\min(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \cdot \hat{A}_t)]$$

where $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{old}(a_t|s_t)$ — ratio of new and old policies

Generalized Advantage Estimation (GAE)

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \cdot \delta_{t+l}^V$$

TD-error δ

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Value network loss + entropy

$$\mathcal{L} = \mathcal{L}_{PPO} - c_1 \cdot \mathbb{E}[(V_\phi(s_t) - R_t)^2] + c_2 \cdot H(\pi_\theta)$$

PPO Hyperparameters: ε (clip range, usually 0.1–0.2) · γ (discount factor, ~0.99) · λ (GAE lambda, ~0.95) · c_1 (value loss coef, ~0.5) · c_2 (entropy coef, ~0.01) · K (ppo epochs, ~4) · batch size (~512)

Models in memory: policy π_θ + reference π_{ref} + value network V_ϕ + reward model r_ψ + old policy π_{old} = 5 models

Problems of RLHF + PPO

Instability PPO is sensitive to hyperparameters ($\epsilon, \gamma, \lambda, LR$). Incorrect settings $\rightarrow$ policy collapse or no progress.

Reward Hacking The model finds loopholes in RM: long answers, keywords, formatting — all to maximize the scalar.

Costliness Three training stages, 5 models in memory (policy, ref, value, RM, old). Huge computational costs.

Distribution Shift π_θ drifts from π_{ref} $\rightarrow$ RM sees OOD answers $\rightarrow$ scores become unreliable. KL penalty doesn't always save.

Manual β Tuning KL coefficient β requires grid search. Small $\beta \rightarrow$ overoptimization, large $\beta \rightarrow$ underfitting. No universal value.

Hence the motivation for DPO and GRPO: Can we remove the separate RM and/or value network while preserving alignment quality? Answer — yes, by reformulating the problem.

DPO

Step 1: The optimal policy can be inverted

$$\pi^*(y|x) \propto \pi_{ref}(y|x) \cdot \exp(r(x,y)/\beta) \Rightarrow r(x,y) = \beta \log(\pi^*(y|x)/\pi_{ref}(y|x)) + \beta \log Z(x)$$

Step 2: Substitute into Bradley-Terry — $Z(x)$ cancels out!

$$P(y_w \succ y_l) = \sigma(\beta \log(\pi(y_w|x)/\pi_{ref}(y_w|x)) - \beta \log(\pi(y_l|x)/\pi_{ref}(y_l|x)))$$

Result: Loss depends only on π_θ and π_{ref} , without explicit r !

What this means:

No need to train a separate reward model, saves time and memory

No need for an RL stage with PPO, ordinary gradient descent

DPO is an offline method: requires a fixed dataset of preference pairs

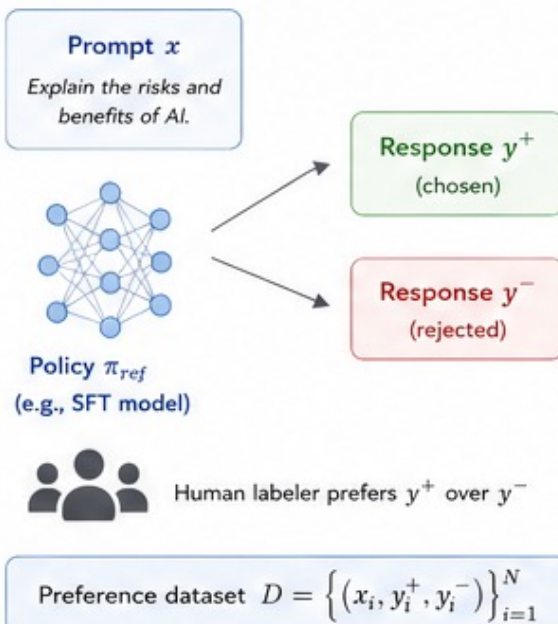
Theoretically equivalent to RLHF with optimal policy

Intuition: Instead of learning an "evaluator" (RM) and then teaching a "generator" (policy) to maximize scores, DPO directly teaches the generator to distinguish good and bad answers.

DPO

1 Collect Data

Sample prompts from a dataset and collect pairwise human preferences between two model responses.



2 Preference Assumption (Bradley-Terry)

Human preference between y^+ and y^- for prompt x is modeled by a Bradley-Terry distribution:

$$P(y^+ \succ y^- | x) = \frac{\exp(s(x, y^+))}{\exp(s(x, y^+)) + \exp(s(x, y^-))}$$

where $s(x, y)$ is a model score (higher is better).

Equivalently, the log-odds is the score difference:

$$\log \frac{P(y^+ \succ y^- | x)}{P(y^- \succ y^+ | x)} = s(x, y^+) - s(x, y^-).$$

DPO parameterizes the score by the difference in log-probabilities under the policy and a reference policy.

3 DPO Objective

Let π_θ be the policy to learn and π_{ref} the reference policy (e.g., SFT model). Define

$$s_\theta(x, y) = \log \pi_\theta(y | x) - \log \pi_{ref}(y | x).$$

The DPO loss for a single preference pair is

$$\mathcal{L}_{DPO}(\theta) = -\log \sigma \left(\beta \left[s_\theta(x, y^+) - s_\theta(x, y^-) \right] \right)$$

$$\text{where } \sigma(z) = \frac{1}{1 + e^{-z}}$$

$\beta > 0$ is an inverse temperature controlling how strongly preferences are enforced.

Intuition

Increase log-prob of chosen (y^+) and/or decrease log-prob of rejected (y^-), relative to the reference, to match human preference.

4 Optimization

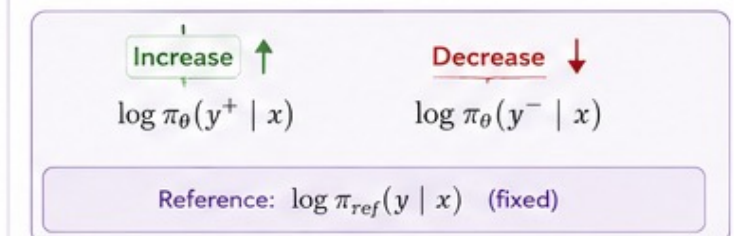
Minimize the average DPO loss over the dataset.

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{(x, y^+, y^-) \sim D} \left[\log \sigma \left(\beta \left[s_\theta(x, y^+) - s_\theta(x, y^-) \right] \right) \right]$$

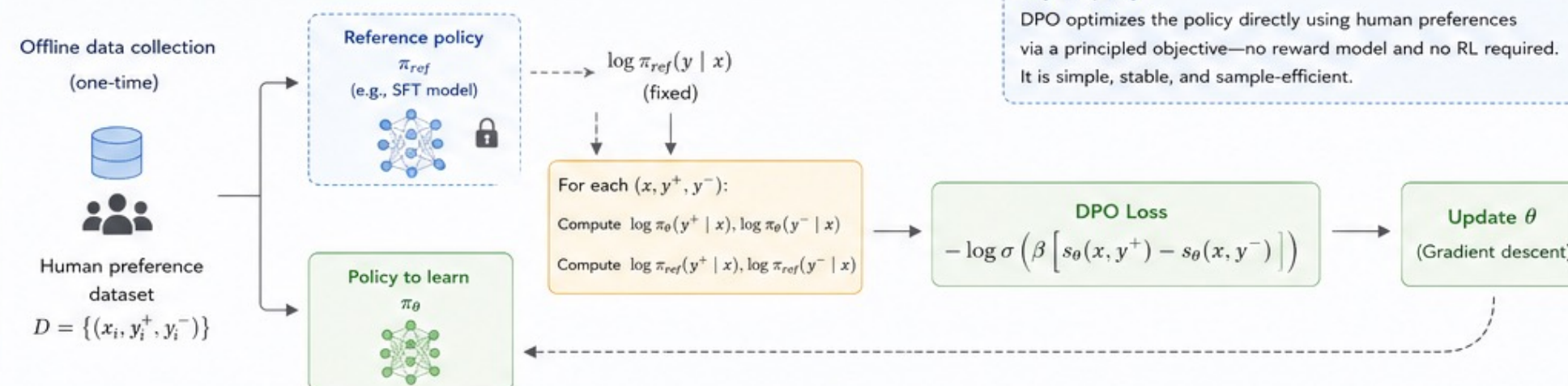
Update θ by gradient descent.

$$\nabla_\theta \mathcal{L}_{DPO}(\theta) = -\mathbb{E}_D \left[(1 - \sigma(z)) \beta \nabla_\theta (s_\theta(x, y^+) - s_\theta(x, y^-)) \right]$$

$$\text{with } z = \beta \left[s_\theta(x, y^+) - s_\theta(x, y^-) \right].$$



5 How DPO Fits in the Alignment Pipeline



Why DPO?

- Direct**
Optimizes the policy directly from preference data.
- No Reward Model**
Avoids training a separate reward model.
- Stable & Efficient**
Uses supervised-style updates; typically more stable and data efficient than RLHF.
- Theoretical Foundation**
Grounded in the Bradley-Terry model of preferences and maximum likelihood.
- Easy to Plug In**
Works with any seq2seq/LLM and existing preference datasets.

Notation

x : prompt
 y^+ : preferred (chosen) response
 y^- : dispreferred (rejected) response
 β : inverse temperature
 π_θ : policy being learned
 π_{ref} : reference policy (fixed)
 $\sigma(z) = 1/(1 + e^{-z})$: logistic sigmoid

Legend

Data Assumption / Theory Objective
Optimization Pipeline / System Fixed / Reference

At a glance

DPO turns pairwise human preferences into a simple, differentiable objective that shifts probability mass from rejected responses to preferred ones relative to a reference model.

DPO Derivation

Starting point: optimal policy

$$\pi^*(y|x) = (1/Z(x)) \pi_{ref}(y|x) \exp(r(x,y)/\beta)$$

Express $r(x,y)$ through π

$$r(x,y) = \beta \log(\pi^*(y|x)/\pi_{ref}(y|x)) + \beta \log Z(x)$$

Substitute into Bradley-Terry (Z cancels!)

$$P(y_w \succ y_l) = \sigma(\beta \log(\pi(y_w|x)/\pi_{ref}(y_w|x)) - \beta \log(\pi(y_l|x)/\pi_{ref}(y_l|x)))$$

MLE on dataset $\rightarrow$ DPO loss

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{\{(x, y_w, y_l) \sim \mathcal{D}\}} [\log \sigma(\beta \log(\pi_\theta(y_w|x)/\pi_{ref}(y_w|x)) - \beta \log(\pi_\theta(y_l|x)/\pi_{ref}(y_l|x)))]$$

DPO Loss Function

DPO Loss Function — Final Form

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{\{(x, y_w, y_l) \sim \mathcal{D}\}} [\log \sigma(\beta \log(\pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x)) - \beta \log(\pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)))]$$

Interpretation

- **Implicit reward** $\hat{r}(x, y) = \beta \log(\pi_{\theta}(y|x)/\pi_{ref}(y|x))$ Loss = binary classification: how well does the model distinguish winner from loser
- **Simplified notation** $\mathcal{L}_{DPO} = -\log \sigma(\beta \cdot (\hat{r}_w - \hat{r}_l))$ where $\hat{r}_w = \log$ -ratio for y_w , $\hat{r}_l = \log$ -ratio for y_l . Training: ordinary gradient descent
- **Numerical Example Given:** $\beta = 0.1, \log \pi_{\theta}(y_w|x) = -2.5, \log \pi_{ref}(y_w|x) = -3.0, \log \pi_{\theta}(y_l|x) = -1.8, \log \pi_{ref}(y_l|x) = -2.0$ Calculation: $\hat{r}_w = 0.1 \times (-2.5 - (-3.0)) = 0.05, \hat{r}_l = 0.1 \times (-1.8 - (-2.0)) = 0.02$
- Loss: $\mathcal{L}_{DPO} = -\log \sigma(0.05 - 0.02) = -\log \sigma(0.03) = -\log(0.5075) \approx 0.678$
- The model correctly prefers y_w , but confidence is low: loss is close to $-\log(0.5) = 0.693$

DPO Advantages

No reward model Saves memory and computation. No need to train and store a separate evaluation model.

No RL stage Simple classification instead of unstable PPO. Ordinary gradient descent.

Single-stage training SFT → DPO, without intermediate RM. Simplifies the development pipeline.

Theoretical equivalence to RLHF At optimal policy, DPO gives the same result as RLHF with optimal RM. Proven in Rafailov et al. (2023).

Good empirical results On instruction and dialogue tasks, DPO is comparable or superior to RLHF+PPO at significantly lower cost.

Key Achievement of DPO Reducing complex RL optimization (3 stages, 5 models, unstable training) to simple supervised learning (1 stage, 2 models, stable gradient descent). This is not an "approximation" of RLHF: it is an equivalent reformulation at optimal policy.

DPO Limitations

Offline limitation Requires fixed preference pairs. Cannot adapt online to new data. Dataset must be collected in advance.

Overoptimization (OOD) On out-of-distribution answers, implicit reward gives unpredictable estimates. Extrapolation beyond data is risky.

Length Bias Tendency toward longer answers: log-prob accumulates over tokens. Long answer = more "confidence."

Sensitivity to β Wrong $\beta \rightarrow$ either underfitting (model doesn't change) or too much deviation from π_{ref} (degradation).

Requires good cold-start Poor SFT model $\rightarrow$ poor candidates in dataset $\rightarrow$ poor alignment. Quality of π_{ref} is critical.

GRPO: Context

Proposed in DeepSeekMath (Shao et al., 2024) for efficient RL training of LLMs on mathematical problems

PPO Problem

- Value network $V(s_t)$ requires:
- Significant memory resources (+1 large model)
- Difficult to train for language tasks
- Needed for every token in the episode
- GAE — additional computations
- Often the main source of instability

GRPO Idea

- Instead of value network — group comparison:
- From one prompt, generate G answers
- Compute rewards for each
- Normalize within the group
- Obtain relative advantage
- Without a separate model!
- $A_i = (r_i - \mu_{group}) / \sigma_{group}$ where $\mu_{group} = (1/G) \sum_j r_j, \sigma_{group} = \sqrt{((1/G) \sum_j (r_j - \mu)^2)}$

GRPO: Algorithm

For each prompt x , perform:

- **Sampling:** $\{y_1, \dots, y_G\} \sim \pi_{old}(\cdot | x)$
- **Evaluation:** $\{r_1, \dots, r_G\} = r_\varphi(x, y_i)$ or heuristic
- **Normalization:** $A_i = (r_i - \mu_{group}) / \sigma_{group}$
- **Update:** PPO-style clipped loss with A_i

Key Properties

- **Zero mean:** $\Sigma A_i = 0$ within the group
- **Unit variance:** $Var(A_i) \approx 1$
- **Robustness:** normalization compensates for reward scale
- **Flexibility:** reward can be anything: trained model, correctness rule, solution accuracy
- **Group size G :** typically 4–16, larger = more stable, but more expensive
- **Why this works:** Instead of teaching the model to predict absolute state value (value network), GRPO uses relative comparison within a group. This is simpler, more stable, and requires no additional parameters.
- **Analogy:** Instead of evaluating "how good is this chess move in absolute units," we compare "is this move better than other moves from the same position."

GRPO: Loss

Clipped policy gradient with group advantages:

$\mathcal{L}_{GRPO}(\theta) = \mathbb{E}[(1/G) \sum_{i=1}^G \min(\rho_i A_i, \text{clip}(\rho_i, 1 - \varepsilon, 1 + \varepsilon) \cdot A_i)] - \beta \cdot KL(\pi_\theta \parallel \pi_{ref})$
where $\rho_i = \pi_\theta(y_i|x) / \pi_{old}(y_i|x)$ — ratio of new and old policies for the i-th answer

KL-penalty is computed via sampling:

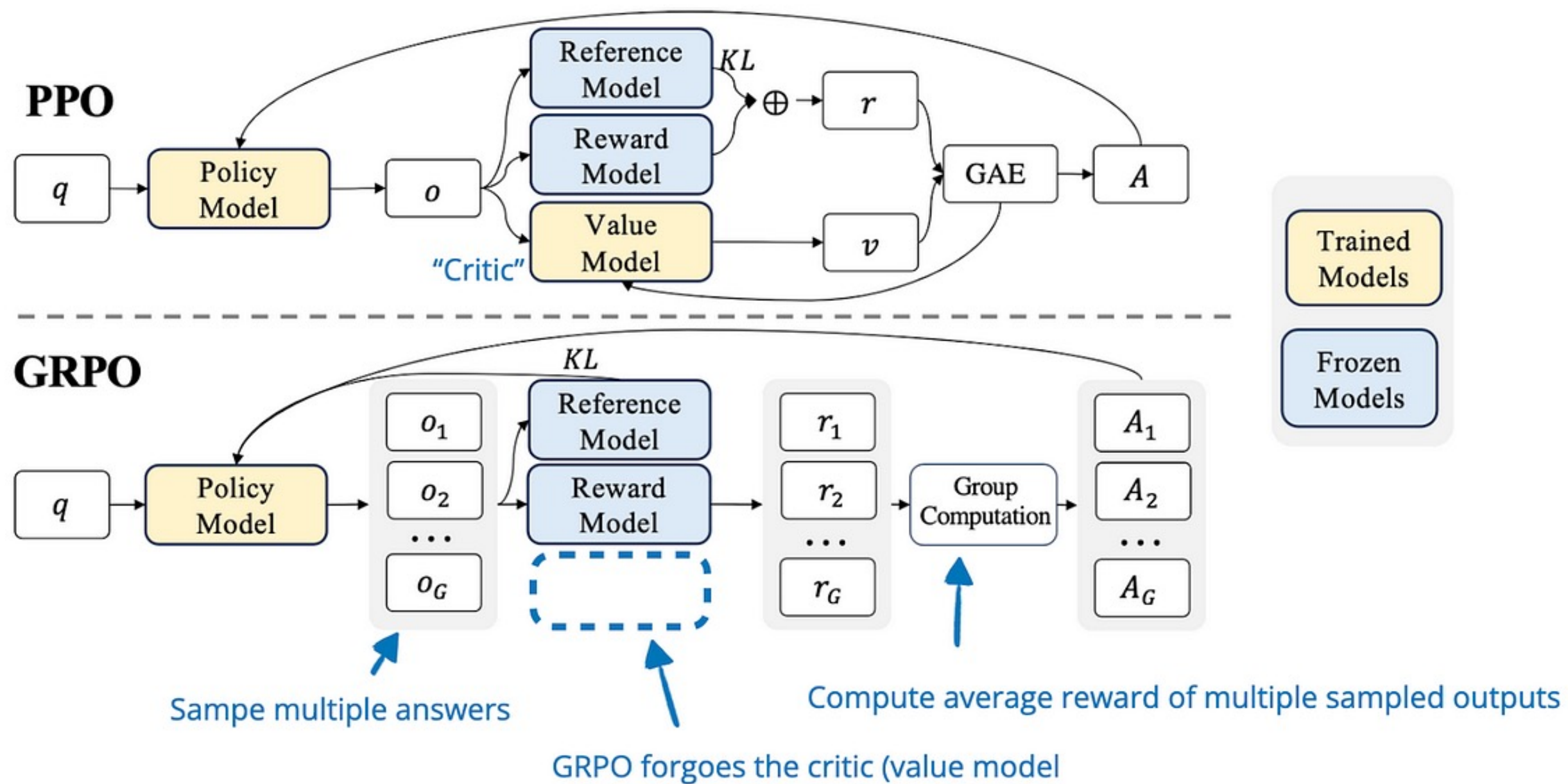
$$KL(\pi_\theta \parallel \pi_{ref}) = \mathbb{E}_{\{x, y \sim \pi_{old}\}} [\log(\pi_{ref}(y|x)/\pi_\theta(y|x)) - \log(\pi_{ref}(y|x)/\pi_{old}(y|x))] \\ = \mathbb{E}[\log(\pi_\theta(y|x)/\pi_{ref}(y|x))]$$

Criterion	PPO	GRPO
Value network	Required -> +1 model	Not required
Advantage	GAE	Group Normalization
Hyperparameters	Many	Fewer
Iteration speed	Slower	Faster
Quality	Good	Comparable or even better

GRPO Advantages:

- **No value network:** Saves ~30-40% GPU memory. Fewer models in memory.
- **Stability:** Group normalization is robust to reward scale.
- **Reward flexibility:** Works with any signal: model, heuristic, ORM.
- **Online mode:** Naturally supports online data collection.

GRPO vs PPO



KTO: Kahneman–Tversky Optimization

Ethayarajh et al. (2024) — KTO: Model Alignment as Prospect Theoretic Optimization

DPO Problem Requires preference pairs (y_w, y_l) — expensive to collect. Needs a labeler to compare two answers.

KTO Solution Works with binary "like/dislike" labels without pairs. Data is significantly cheaper.

Theory: Kahneman-Tversky Prospect Theory

Gain/loss asymmetry: people perceive positive and negative outcomes differently.

KTO loss accounts for this asymmetry through different weights for desirable and undesirable answers.

KTO Loss Function

$$\mathcal{L}_{KTO} = \mathbb{E}_{x,y \sim D} [\lambda_y - v(x, y)], \text{ where } v(x, y) = \lambda_y (\sigma(\beta * \text{sign} * (r(x, y) - z_{ref})))$$

where $\lambda_y = \lambda_D$ for desirable (good) answers, λ_U for undesirable (bad),
 $z_{ref} = KL(\pi_\theta(y'|x) \parallel \pi_{ref}(y'|x))$ – reference point.

Key advantage: No pairs needed, binary labels are sufficient.

IPO: Identity Preference Optimization

Azar et al. (2023) — A General Theoretical Paradigm for Learning from Preferences (IPO)

DPO Problem At large β , DPO can overfit — implicit reward becomes too confident even on training data, leading to overoptimization.

IPO Solution Adds regularization based on log-difference. Quadratic loss instead of logistic — prevents excessive confidence.

IPO Loss Function

$$\mathcal{L}_{IPO} = \mathbb{E} [(\log(\pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x)) - \log(\pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)) - \tau^{-1})^2]$$

Difference from DPO

DPO: $-\log \sigma(\beta \cdot (\hat{r}_w - \hat{r}_l))$ — logistic loss, can give infinitely large values for confident predictions

IPO: $(\hat{r}_w - \hat{r}_l - \tau^{-1})^2$ — quadratic loss, "penalizes" for too large a difference, more stable at large β

τ (temperature): controls desired difference between $\hat{r}_w$ and $\hat{r}_l$. Smaller τ = more regularization.

Method Selection

Criterion	DPO	GRPO	RLHF + PPO
Pairs needed	Yes	Yes / Groups	Yes
Stability	High	High	Medium
GPU Memory	Low	Medium	High
Code complexity	Low	Medium	High
Online learning	No	Yes	Yes
Scalability	Good	Good	Limited
Best scenario	Static dataset	Online + generation	Max. flexibility

Quality vs Cost

Key Conclusions

RLHF+PPO: High costs, good quality with proper tuning, but unstable

DPO: Low costs, good quality on static data, fast launch

GRPO: Medium costs, excellent quality for tasks with clear reward (math, coding)

KTO: Lowest data costs, comparable quality

Trend: Pipeline simplification without quality loss DPO and GRPO show that comparable quality can be achieved with lower costs.

Practice

Choosing β and the Answer Length Problem

Parameter β (KL-coefficient)

Small β (~ 0.1): $\rightarrow$ Strong alignment, low diversity $\rightarrow$ Risk of overoptimization and degradation

Large β (~ 0.5 – 1.0): $\rightarrow$ Weak alignment, high diversity $\rightarrow \pi_\theta \approx \pi_{\text{ref}}$, minimal changes

Recommendation: Start with $\beta = 0.1$ – 0.3

Length Bias in DPO

Problem: The model prefers longer answers, log-prob accumulates over tokens

Solutions:

- Normalization: divide log-ratio by $|y|$
- Length penalty in loss
- Controlled decoding with length limit

Normalized DPO Loss (length bias fix)

$$\mathcal{L}_{DPO_{\text{norm}}} = -\log \sigma \left(\beta \cdot \left((1/|y_w|) \log(\pi_\theta(y_w)/\pi_{\text{ref}}(y_w)) - (1/|y_l|) \log(\pi_\theta(y_l)/\pi_{\text{ref}}(y_l)) \right) \right)$$

Practical Advice: Always start with normalized DPO loss when working with variable-length answers. This eliminates $\sim 80\%$ of length bias without additional hyperparameters. Evaluate the trade-off between alignment and diversity on the validation set.

On-p. vs Off-p. and Cold-start

DPO: Off-policy

- Trains on a fixed dataset
- Suffers from distribution shift
- When π_θ deviates strongly from data $\rightarrow$ implicit reward is unreliable
- Does not adapt to new patterns

GRPO: On-policy

- Each iteration generates new answers
- Under current policy π_θ
- Naturally adapts to shift
- Data is always "fresh"

Importance of Cold-start

- SFT Model All methods assume a good π_{ref} , an SFT model capable of generating diverse candidates.
- Poor SFT $\rightarrow$ poor candidates for preferences $\rightarrow$ poor alignment regardless of method
- Good SFT $\rightarrow$ good candidates $\rightarrow$ alignment improves ranking of already good answers

Recommendation: Invest in SFT quality before alignment. This is the foundation: alignment only ranks, it does not create quality from scratch.

Mixing with SFT

Mixing alignment loss with SFT loss

$$\mathcal{L}_{total} = \mathcal{L}_{alignment} + \alpha \cdot \mathcal{L}_{SFT} \text{ where } \mathcal{L}_{SFT} = -\mathbb{E}[\log \pi_{\theta}(y_{good}|x)]$$

This prevents "forgetting" good patterns from SFT data. Typically $\alpha = 0.1-0.5$.

Method Selection Recommendations

- **DPO** Have a good pair dataset → simple, fast, quality
- **GRPO** Need online training with generation → flexibility, stability
- **RLHF+PPO** Maximum flexibility and sufficient resources → control of all components
- **KTO** Only binary labels → cheap data, good quality
- **IPO** DPO overfits → more stable at large β

Current Research

Multi-task Alignment How to align a model on multiple criteria simultaneously (safety, helpfulness, honesty) without degradation?

Constitutional AI Self-improvement through principle-based criticism (Bai et al., 2022). The model criticizes its own answers based on a set of rules.

Active Preference Learning Minimizing the number of queries to human labelers while maximizing information about preferences. Choosing the most informative pairs.

Multimodal Alignment Video, images, audio: how to collect and use preferences for multimodal models?

Self-improvement without Humans RL "without RLHF": the model learns from its own reasoning: STaR (Self-Taught Reasoner), Self-Rewarding LM. Can a model improve independently without human feedback?

Open Problems

Theoretical Guarantees Why DPO/GRPO work well in practice: insufficient theory. Boundaries of applicability?

Scaling Stability and efficiency of alignment on models with hundreds of billions of parameters.

Composite Preferences How to combine preferences of different user groups without conflicts?

Long-term Effects Impact of alignment on model knowledge and reasoning. An "aligned" model may lose factual knowledge or ability for complex reasoning.

Bypassing Restrictions (Jailbreaks) Adversarial attacks on aligned models. Why doesn't alignment guarantee safety? How to create robust alignment?

From RLHF to DPO/GRPO

Key Trend: Pipeline simplification without quality loss

2020-2022	2023	2024
RLHF + PPO	DPO	GRPO
3 training stages	1 training stage	Group advantage
Complex, expensive	Simple, stable	No value network
Unstable	Offline	Online + offline
5 models in memory	2 models in memory	3 models in memory

Main Insight:

Alignment is not necessarily RL. Sometimes a smart reformulation of the problem is sufficient:

- DPO showed that RLHF can be reduced to supervised learning
- GRPO showed that the value network can be replaced with group comparison
- KTO showed that preference pairs are not necessary

Future: Even simpler methods, self-improvement, multimodality

Fin.

Thanks for you Attention!

(because Attention Is All I Need)

